

# SPACE INVADERS

Programación Orientada a Objetos (Java)

**Sprint: Demo Day + entrega en eGela**

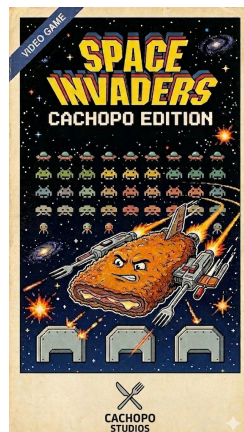
*Documentación final*

**Grupo: Cachopin**

11 de mayo de 2026

Estructura según entrega final (Apartados a, b, c, f, g, h).

Los informes de MVC/patrones y de uso de LLM se entregan como documentos aparte.



# Índice

---

|                                                            |          |
|------------------------------------------------------------|----------|
| <b>1. Introducción: presentación y objetivos</b>           | <b>2</b> |
| 1.1. Presentación del proyecto . . . . .                   | 2        |
| 1.2. Objetivos . . . . .                                   | 2        |
| <b>2. Herramientas LLM y asistentes de IA</b>              | <b>2</b> |
| 2.1. Herramientas utilizadas . . . . .                     | 2        |
| 2.2. Uso en el proyecto . . . . .                          | 2        |
| 2.3. Generación de imágenes . . . . .                      | 2        |
| <b>3. Gestión: reparto de tareas, reuniones y recursos</b> | <b>3</b> |
| 3.1. Metodología . . . . .                                 | 3        |
| 3.2. Equipo . . . . .                                      | 3        |
| <b>4. Desarrollo: objetivos y decisiones por sprint</b>    | <b>3</b> |
| 4.1. Sprint 1 . . . . .                                    | 3        |
| 4.2. Sprint 2 . . . . .                                    | 4        |
| 4.3. Sprint 3 . . . . .                                    | 4        |
| <b>5. Reparto de tareas por sprint</b>                     | <b>4</b> |
| 5.1. Sprint 1 . . . . .                                    | 4        |
| 5.2. Sprint 2 . . . . .                                    | 5        |
| 5.3. Sprint 3 y cierre ( <i>entrega final</i> ) . . . . .  | 5        |
| <b>6. Conclusiones: reflexión y cambios futuros</b>        | <b>6</b> |
| 6.1. Reflexión . . . . .                                   | 6        |
| 6.2. Cambios y mejoras futuras . . . . .                   | 6        |

## Introducción: presentación y objetivos

---

### Presentación del proyecto

Space Invaders es un proyecto académico en Java que reproduce el juego clásico: una nave en una cuadrícula debe eliminar enemigos que descienden sin ser alcanzada. El trabajo se ha organizado en tres sprints con entregas incrementales y revisión tras cada uno.

### Objetivos

Aplicar POO en equipo, implementar patrones de diseño (MVC+Observer, Factory, Strategy, Composite), trabajar con metodología ágil y entregar un producto jugable y bien documentado.

## Herramientas LLM y asistentes de IA

---

En el desarrollo moderno de software, los **modelos de lenguaje de gran escala (LLM)** son herramientas que pueden mejorar la productividad y la velocidad de desarrollo. Durante *Space Invaders* se utilizaron varias herramientas de este tipo.

Es importante aclarar que la **responsabilidad final del código y del diseño recae en el equipo de desarrollo**: los LLM y los asistentes del IDE fueron **complementarios**, no sustitutos del criterio humano ni de la implementación supervisada.

### Herramientas utilizadas

Visual Studio Code + GitHub Copilot, Cursor, Gemini y Claude para asistencia durante el desarrollo.

### Uso en el proyecto

Las herramientas más usadas fueron los IDE con asistente de IA: autocompletado para código repetitivo, adaptación de clases, formato de documentos. La lógica compleja del juego y todas las sugerencias fueron supervisadas antes de integrar.

### Generación de imágenes

Se utilizó Gemini para generar los fondos del juego con temática cachopo-nave en estilo pixel art.



Figura 1: Fondo del juego generado con Gemini: temática *cachopo-nave* en estilo pixel art.

## Gestión: reparto de tareas, reuniones y recursos

---

### Metodología

Se ha seguido un esquema ágil: sprints de 2-3 semanas, demos y revisión con el cliente (profesorado) tras cada entrega. Uso de Git para versionado del código, reuniones de equipo y integración frecuente.

### Equipo

El trabajo se ha distribuido entre cuatro miembros: **Unai Urrutia**, **Ekain Ortega**, **Eder Lorenzo** y **Asier López**. Cada uno ha contribuido en modelo, vista, controlador, integración y documentación según las necesidades de cada sprint.

## Desarrollo: objetivos y decisiones por sprint

---

### Sprint 1

#### Objetivos principales:

- Tener un **juego jugable mínimo**: nave, enemigo(s), disparo y condiciones de fin de partida.
- Establecer la **arquitectura base** del código y la división en capas de forma coherente.
- Fijar convenciones de paquetes, nombres y flujo de trabajo en Git.

#### Decisiones clave:

- Matriz de  $10 \times 10$  píxeles como tamaño del tablero de juego.
- Uso de `Timer` para movimiento de enemigos y disparos.
- Naves y enemigos como entidades multipixel usando `Composite`.

## Sprint 2

### Objetivos principales:

- **Ampliar** funcionalidad: distintas naves, enemigos y disparos según requisitos del backlog.
- Refactorizar donde haga falta para mantener el código **mantenible** sin romper lo ya entregado.

### Decisiones clave:

- Patrón **Factory** para creación centralizada de naves (Nave1, Nave2, Nave3, Nave4).
- Patrón **Strategy** para tipos de disparo (Pixel, Flecha, Rombo, Doble).
- Patrón **Composite** para geometría variable de naves y enemigos.

## Sprint 3

### Objetivos principales:

- Cerrar **historias de usuario** pendientes, pulir la experiencia de juego y la estabilidad.
- Completar **documentación final**, preparar **Demo Day** y entrega en eGela.
- Opcional: extensiones acordadas para la nota máxima (HU adicional), si aplica.

### Decisiones clave:

- Sistema de puntuación visible en pantalla.
- Pantalla final (FinalFrame) con opción de reinicio desde menú.
- Fondos de imagen para todas las pantallas.

## Reparto de tareas por sprint

---

### Sprint 1

**Sprint 1** (juego inicial y MVC+Observer): **50 h** totales según documento de reparto.

| Tarea                                                              | h         | UU           | EO       | EL         | AL           |
|--------------------------------------------------------------------|-----------|--------------|----------|------------|--------------|
| MVC e ideamiento de clases principales                             | 5         | 2,5          | 0,5      | 0,5        | 1,5          |
| Repositorio Git y configuración inicial                            | 1         | 0,3          | 0        | 0          | 0,7          |
| Clases principales del modelo                                      | 5         | 2,5          | 0,5      | 0,5        | 1,5          |
| Lógica de juego, disparo y colisiones                              | 4         | 1,5          | 0,5      | 0,5        | 1,5          |
| Vista, controlador y métodos <code>update</code>                   | 3         | 1            | 0,5      | 0,5        | 1            |
| <code>notify/update</code> para inicializar <code>MainFrame</code> | 4         | 1,5          | 0,25     | 0,25       | 1,5          |
| Corrección colisiones, límites, movimiento                         | 3         | 1            | 0,5      | 0,5        | 1            |
| Refinamiento del modelo                                            | 1         | 0,25         | 0,25     | 0,25       | 0,25         |
| <code>MainFrame</code> con rejilla de <code>JLabel</code>          | 5         | 2            | 1        | 0,5        | 1,5          |
| Reestructuración <code>update/notify</code> en vista               | 5         | 2            | 0,5      | 0,5        | 2            |
| Revisión patrón MVC+Observer y fin de partida                      | 5         | 2            | 0,5      | 0,5        | 2            |
| Documentación                                                      | 6         | 2            | 1        | 1          | 2            |
| Diagramas UML y MVC+Observer                                       | 3         | 1            | 1        | 1          | 0            |
| <b>Suma tiempo (tareas)</b>                                        | <b>50</b> |              |          |            |              |
| <b>Suma ded. por miembro</b>                                       |           | <b>19,55</b> | <b>7</b> | <b>6,5</b> | <b>16,45</b> |

*Sprint 1 — columnas de personas: **UU**/**EO**/**EL**/**AL** = Unai Urrutia, Ekain Ortega, Eder Lorenzo y Asier López. En las tablas Sprint 2 y Sprint 3 no se incluye columna de Eder Lorenzo (en los documentos de reparto su dedicación en esas fases es nula). Al sumar las ded. de Sprint 1, el documento de reparto cuadra en cada fila con la columna **h**, pero los totales personales suman 49,5 h (0,5 h de diferencia respecto al total de tareas de 50 h).*

## Sprint 2

**Sprint 2** (patrones Factory, Strategy, Composite): **50 h** reales. Resumen estadístico del equipo según `REPARTO TAREAS SPRINT2.tex`: estimado 27,2 h; real por miembro Unai 16 h, Ekain 16,5 h, Asier 17,5 h.

| Tarea                                                  | h         | UU        | EO          | AL          |
|--------------------------------------------------------|-----------|-----------|-------------|-------------|
| Patrón Factory ( <code>NaveFactory</code> , Singleton) | 4         | 0,5       | 2           | 1,5         |
| Patrón Strategy ( <code>StrategyDisparo</code> )       | 8         | 1         | 3           | 4           |
| Patrón Composite (píxeles jerárquicos)                 | 6         | 2         | 2           | 2           |
| Actualización diagrama UML                             | 8         | 6         | 1           | 1           |
| Tres naves ( <code>Nave1</code> – <code>Nave3</code> ) | 5         | 2         | 1           | 2           |
| Naves multipixel ( <code>construir</code> , geometría) | 6         | 1         | 2,5         | 2,5         |
| Enemigos multipixel y <code>FlotaEnemigos</code>       | 3         | 0,5       | 1           | 1,5         |
| Tipos de disparo (Pixel, Flecha, Rombo)                | 5         | 2         | 1           | 2           |
| Documentación de patrones                              | 5         | 1         | 3           | 1           |
| <b>Total por miembro</b>                               | <b>50</b> | <b>16</b> | <b>16,5</b> | <b>17,5</b> |

## Sprint 3 y cierre (*entrega final*)

**Sprint 3** (puntuación, `FinalFrame`, fondos, `Nave4`, `DisparoDoble`): **25 h** reales según `REPARTO TAREAS SPRINT3.tex`. Esta fase coincide con las mejoras y la preparación documental/de demo de la entrega final.

| Tarea                                           | h         | UU       | EO         | AL         |
|-------------------------------------------------|-----------|----------|------------|------------|
| Sistema de puntuación en <code>MainFrame</code> | 7         | 3,5      | 1          | 2,5        |
| <code>FinalFrame</code> : victoria / game over  | 5         | 1        | 2,5        | 1,5        |
| Reiniciar juego desde <code>FinalFrame</code>   | 4         | 1        | 2          | 1          |
| Imágenes de fondo en todas las pantallas        | 5         | 1,5      | 1,5        | 2          |
| Estrategia <code>DisparoDoble</code>            | 2         | 1        | 0          | 1          |
| <code>Nave4</code> con disparo doble            | 2         | 1        | 0,5        | 0,5        |
| <b>Total por miembro</b>                        | <b>25</b> | <b>9</b> | <b>7,5</b> | <b>8,5</b> |

## Conclusiones: reflexión y cambios futuros

---

### Reflexión

El proyecto ha permitido al equipo experimentar con programación orientada a objetos en un entorno colaborativo, aplicando patrones de diseño reales y metodología ágil. Los mayores desafíos fueron la integración entre patrones (Composite + Strategy), la coordinación de cambios simultáneos en Git y mantener la calidad del código conforme crecía en complejidad.

### Cambios y mejoras futuras

Líneas de mejora futuras:

- Pruebas unitarias (JUnit) para la lógica del modelo.
- Niveles de dificultad progresiva.
- Sonido y animaciones.
- Optimización de rendimiento si la complejidad del juego aumenta significativamente.